

Nova Voice Agent — Análisis Arquitectónico Enterprise Completo

Evolución Profesional hacia Enterprise-Ready

Fecha: 2026-05-19 | Autor: Principal AI Architect

Índice

- 1. Análisis Arquitectónico Completo
- 2. Optimización Profesional de IA
- 3. Sistema de Memoria Avanzado
- 4. Voice AI Profesional
- 5. Detección Emocional y Comportamiento Adaptativo
- 6. Arquitectura RAG Empresarial
- 7. Tool Calling y Multi-Agent Systems
- 8. Escalabilidad Empresarial
- 9. Seguridad para IA Conversacional
- 10. Sistema de Métricas y Observabilidad
- 11. Optimización de Costos
- 12. Arquitectura Final Ideal
- 13. Roadmap Técnico Profesional
- 14. Conclusión Técnica Profesional

1. Análisis Arquitectónico Completo

1.1 Fortalezas Actuales

Área	Fortaleza	Impacto
Modularidad	Separación clara: <code>ai/</code> , <code>core/</code> , <code>telephony/</code> , <code>actions/</code> , <code>database/</code>	Facilita extensión
Event-Driven	<code>EventBus</code> desacopla componentes	Bajo coupling
Dual-Input	Soporta WebSocket (browser) y AudioSocket (Asterisk)	Flexibilidad
Function Registry	Patrón Registry para tool-calling dinámico	Extensible
Prompt Builder	Sistema configurable de personalidad con builder visual	UX admin
FTS5	Full-text search con fallback tokenizado	Búsqueda robusta
Async-first	Todo el stack es <code>asyncio</code> -native	Rendimiento

1.2 Debilidades Críticas

Coupling en `main.py` — Orquestación Monolítica

```
main.py actúa como God Object:  
- Instancia TODOS los servicios  
- Inyecta dependencias manualmente via globals  
- Gestiona WebSocket, lifecycle, routing  
- NO hay Dependency Injection container
```

Riesgo: Cualquier cambio en un servicio requiere modificar `main.py`. Imposible testear componentes aislados.

Inyección de Dependencias via Globals

```
# Patrón actual (ANTI-PATTERN):  
_db: DatabaseManager | None = None  
def set_db(db): global _db; _db = db
```

Impacto: Race conditions potenciales, imposible paralelizar tests, acoplamiento implícito.

Single-Process Architecture

```
PROCESO ÚNICO  
FastAPI + AudioSocket + Gemini + DB  
(todo en el mismo event loop)
```

Riesgo: Un bloqueo en Gemini afecta AudioSocket. SQLite no soporta writes concurrentes. Un crash = caída total.

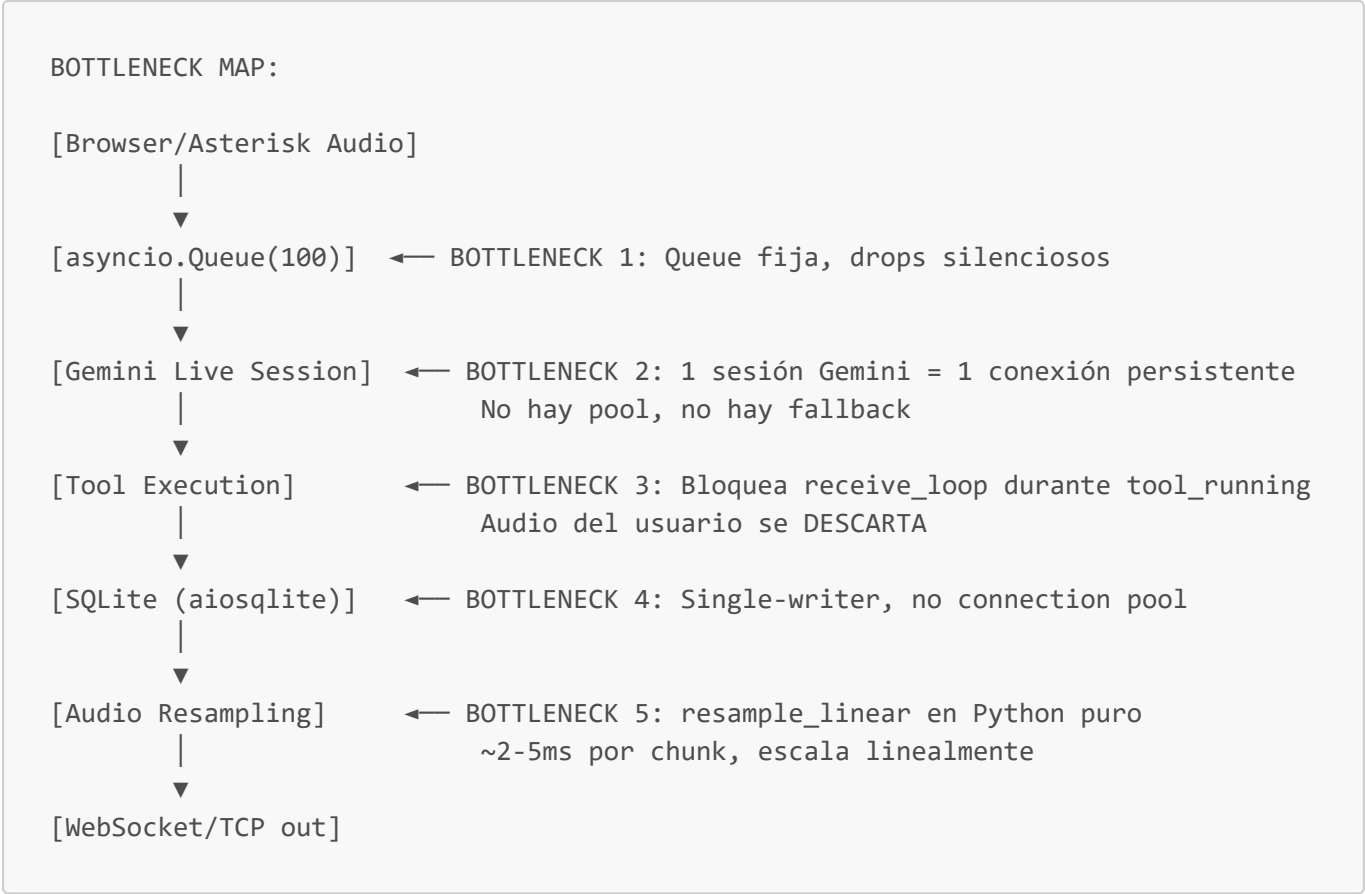
Sin Circuit Breaker ni Retry

- Si Gemini falla → sesión muere silenciosamente
- Si la DB falla → error sin recovery
- No hay health checks internos entre componentes

Sin Observabilidad

- Solo `loguru` (logs texto)
- No hay métricas (latencia, tokens, costos)
- No hay tracing distribuido
- No hay alerting

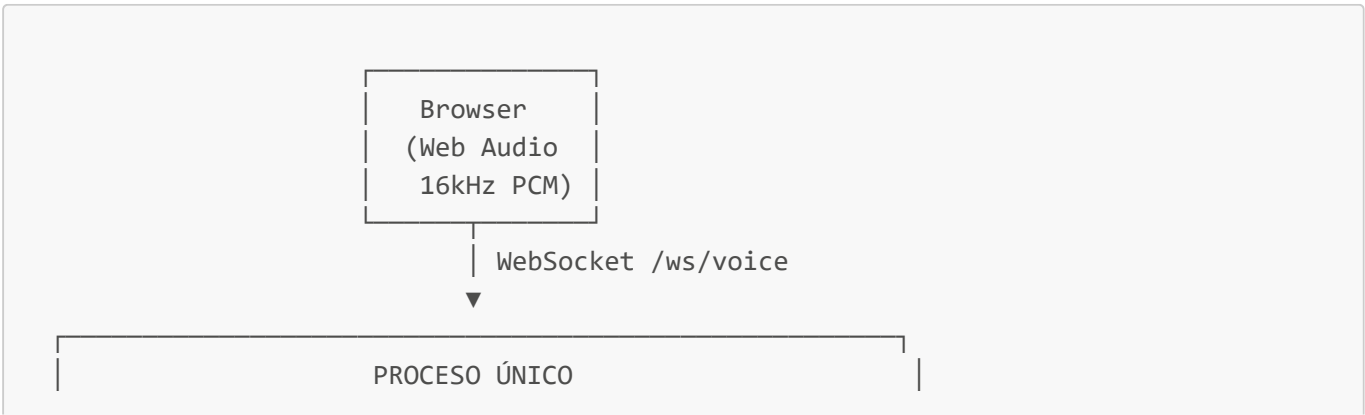
1.3 Cuellos de Botella

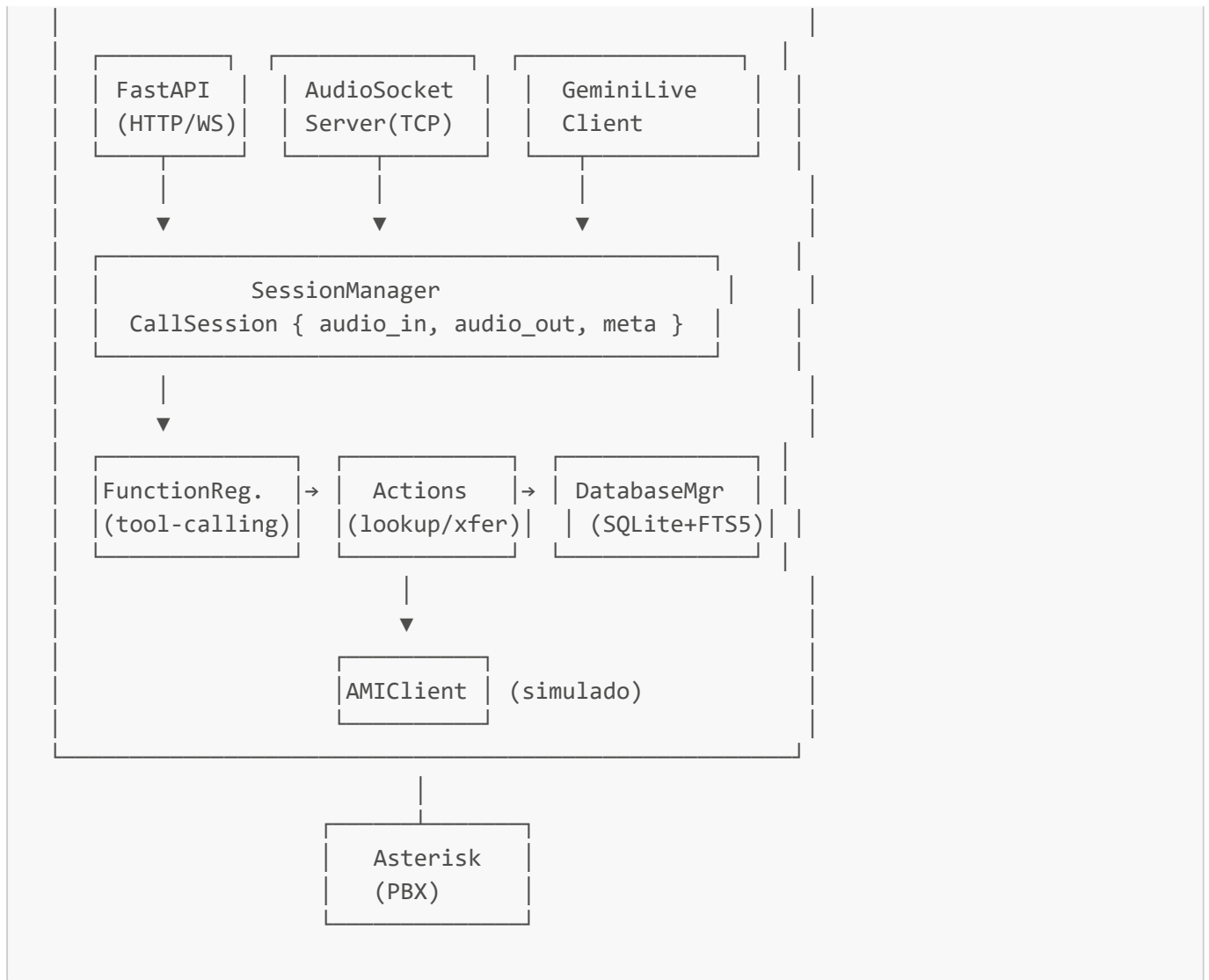


1.4 Riesgos Técnicos en Producción

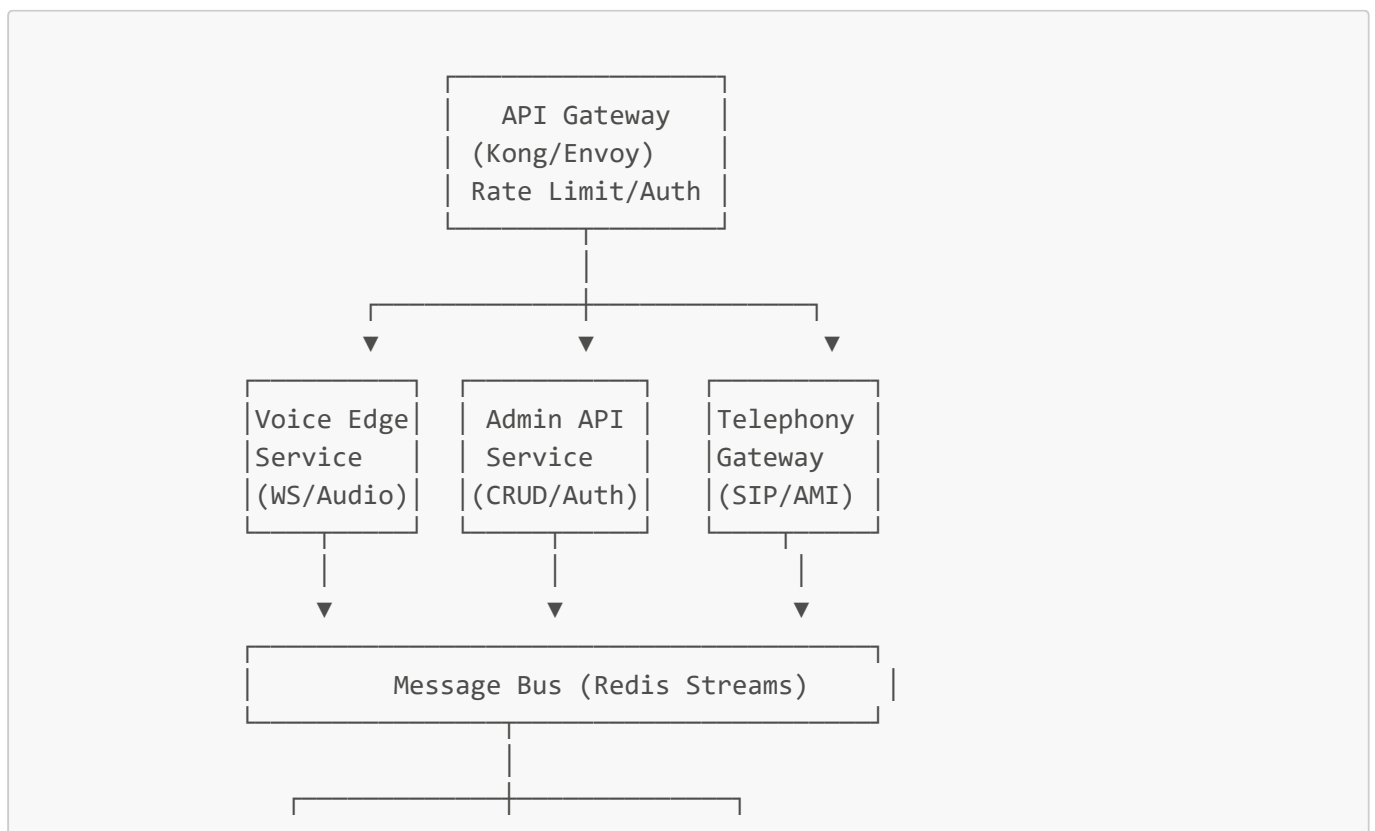
Riesgo	Severidad	Descripción
API Key expuesta	● CRÍTICO	.env contiene GEMINI_API_KEY en texto plano, sin rotación
Sin auth en API admin	● CRÍTICO	/api/admin/* completamente abierto (auth/ está vacío)
SQLite en producción	● ALTO	Límite de ~50 writes/segundo, sin replicación
Sin rate limiting	● ALTO	Abuso potencial de WebSocket y API
AMI simulado	● MEDIO	Transferencias no funcionan en producción real
Sin HTTPS/WSS	● ALTO	Audio transmitido en texto plano
Sin graceful shutdown	● MEDIO	Sesiones activas se cortan abruptamente

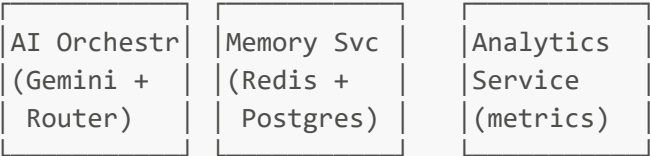
1.5 Diagrama de Arquitectura Actual





1.6 Arquitectura Ideal Propuesta (Preview)





2. Optimización Profesional de IA

2.1 Problema Actual: Dependencia Total de Gemini

FLUJO ACTUAL (todo va a Gemini):

"¿Cuál es el horario?" → Gemini Live (\$\$\$\$)

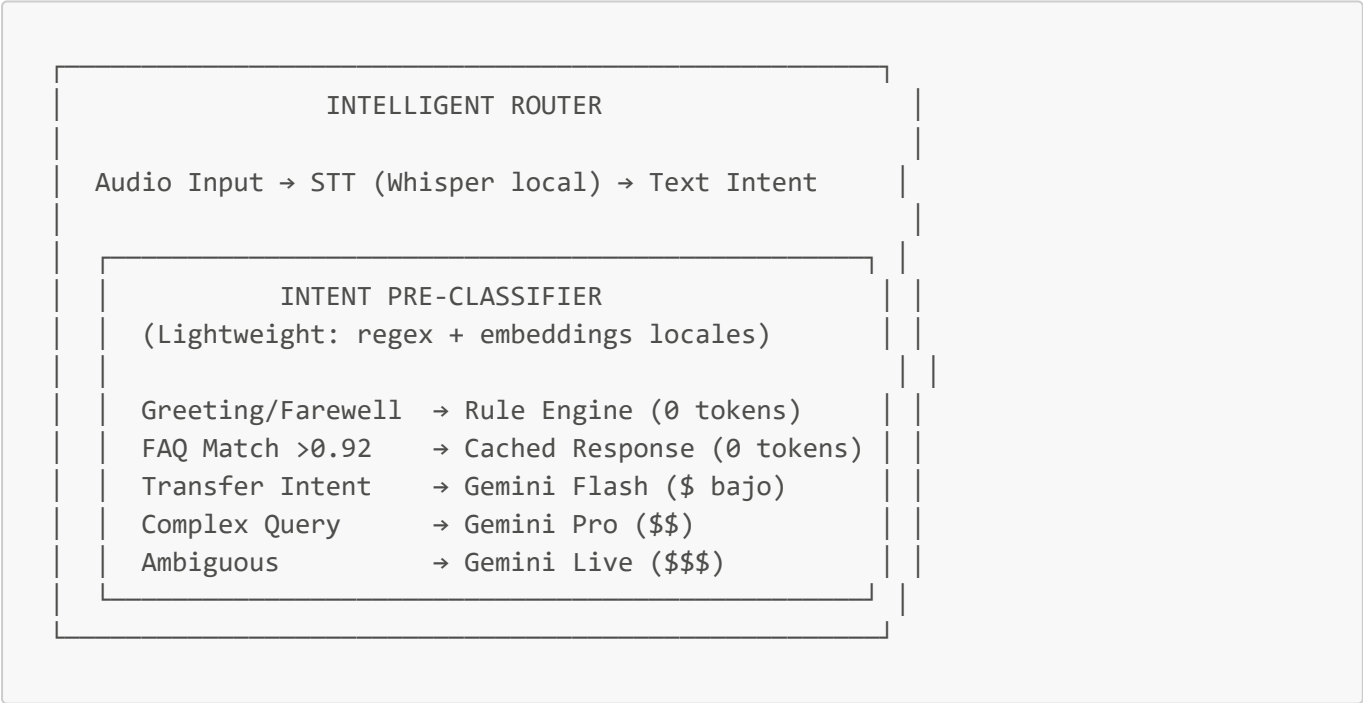
"Transfiere a ventas" → Gemini Live (\$\$\$\$)

"Hola" → Gemini Live (\$\$\$\$)

"Adiós" → Gemini Live (\$\$\$\$)

Costo estimado actual: ~\$0.15-0.40 USD por llamada de 3 min (audio bidireccional + tool calls).

2.2 Arquitectura de IA Híbrida Propuesta



Cuándo usar cada tier:

Tier	Modelo	Cuándo	Costo	Latencia
T0	Reglas/Regex	Saludos, despedidas, confirmaciones sí/no	\$0	<1ms
T1	Embeddings locales	FAQ match, intent classification	\$0	<10ms

Tier	Modelo	Cuándo	Costo	Latencia
T2	Gemini Flash	Extracción de entidades, queries simples	~\$0.001	~200ms
T3	Gemini Live	Conversación compleja, audio nativo	~\$0.05/min	~500ms

Implementación del Semantic Router:

```
class SemanticRouter:
    """Pre-clasifica intent antes de enviar a modelo grande."""

    RULE_INTENTS = {
        r"(hola|buenos?\s*d[ií]as|buenas?\s*tardes)": "greeting",
        r"(adi[oó]s|hasta\s*luego|bye|chao)": "farewell",
        r"(s[ií]|no|correcto|exacto)": "confirmation",
    }

    async def route(self, text: str) -> RoutingDecision:
        # T0: Reglas determinísticas
        for pattern, intent in self.RULE_INTENTS.items():
            if re.match(pattern, text, re.I):
                return RoutingDecision(tier=0, intent=intent)

        # T1: Semantic similarity contra FAQ embeddings
        embedding = await self.embed(text)
        match = self.faq_index.search(embedding, threshold=0.92)
        if match:
            return RoutingDecision(tier=1, cached_response=match.answer)

        # T2/T3: Clasificación por complejidad
        complexity = await self.classify_complexity(text)
        if complexity < 0.5:
            return RoutingDecision(tier=2, model="gemini-flash")
        return RoutingDecision(tier=3, model="gemini-live")
```

Impacto estimado: Reducción del 40-60% en costos de tokens. 70% de interacciones rutinarias se resuelven en T0/T1.

2.3 Token Reduction Pipeline

PIPELINE DE REDUCCIÓN:

1. PROMPT COMPRESSION

System prompt actual: ~800 tokens
Optimizado: ~400 tokens (50% reducción)
Técnica: Eliminar redundancias, usar formato compacto

2. CONTEXT WINDOW MANAGEMENT

Actual: Gemini Live mantiene todo el historial
Propuesto: Sliding window de 10 turnos + resumen comprimido

3. TOOL RESPONSE MINIMIZATION

Actual: Devuelve TODOS los campos de producto

Propuesto: Solo campos relevantes al query

Ejemplo:

ANTES:

```
{"product_name": "..", "description": "..", "price": .., "stock": .., "category": "..", "brand": "..", "color": "..", "weight": ".."}
```

DESPUÉS: {"name": "..", "price": .., "stock": ..} (60% menos tokens)

4. SEMANTIC CACHING

Cache responses para queries frecuentes

Key: embedding del query (no texto exacto)

TTL: 1 hora para inventario, 5 min para extensiones

Hit rate esperado: 25-35%

2.4 Dynamic Prompt Assembly

```
class DynamicPromptAssembler:
    """Ensambla prompt mínimo según contexto de la conversación."""

    def assemble(self, session: CallSession) -> str:
        sections = [self.CORE_IDENTITY] # Siempre (~100 tokens)

        # Solo incluir capabilities relevantes al intent detectado
        if session.detected_domain == "inventory":
            sections.append(self.INVENTORY_INSTRUCTIONS)
        elif session.detected_domain == "directory":
            sections.append(self.DIRECTORY_INSTRUCTIONS)

        # Solo incluir reglas activadas
        if session.turn_count > 5:
            sections.append(self.CONCISENESS_RULE)

        if session.frustration_score > 0.6:
            sections.append(self.EMPATHY_ESCALATION)

        return "\n".join(sections)
```

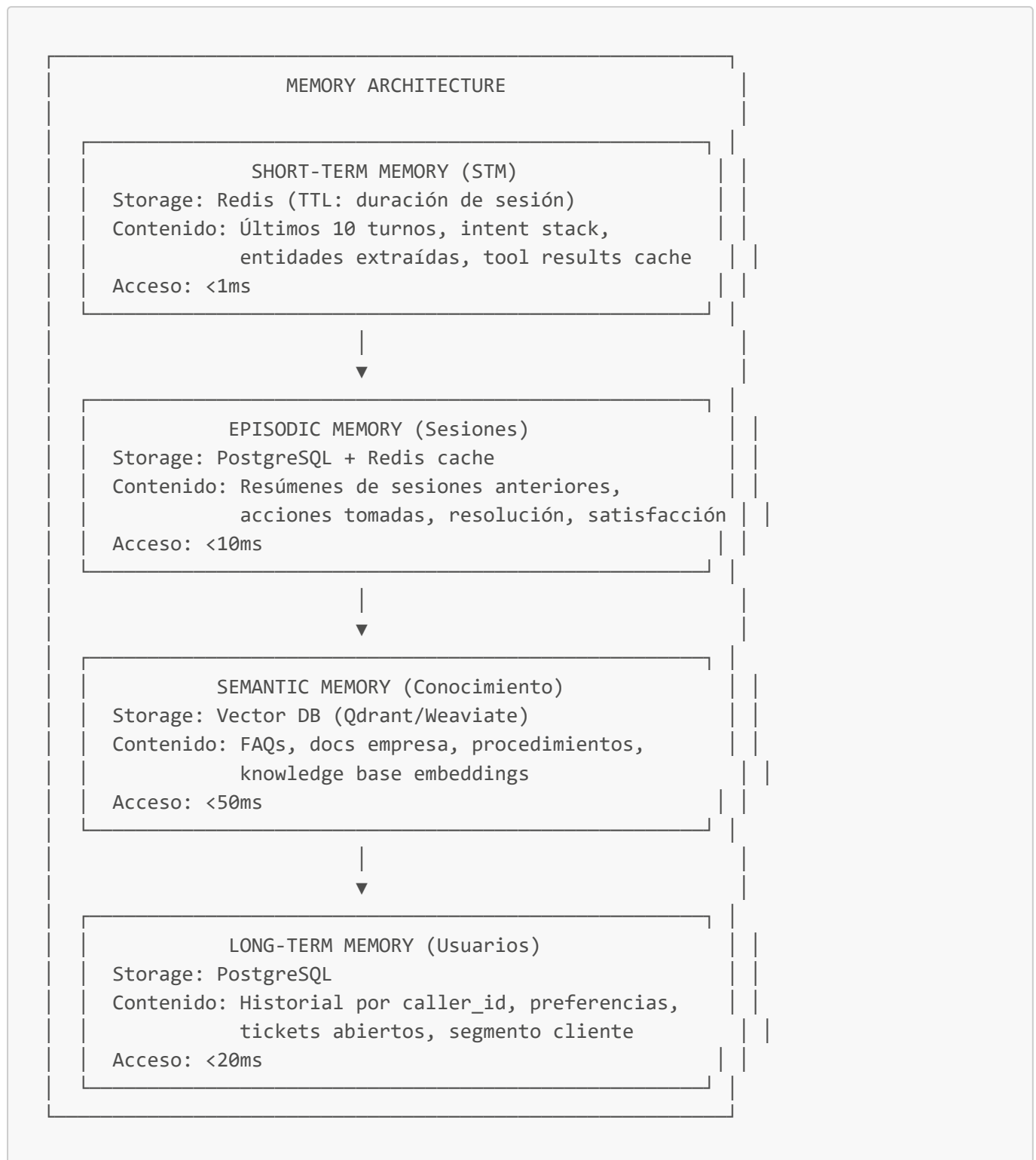
3. Sistema de Memoria Avanzado

3.1 Problema Actual

- **No hay memoria persistente** — cada sesión empieza de cero
- Gemini Live mantiene contexto intra-sesión, pero se pierde al desconectar
- No hay resumen de conversaciones previas

- `call_logs` almacena pero nunca se recupera para contexto

3.2 Arquitectura de Memoria Propuesta



3.3 Memory Retrieval Pipeline

```

class MemoryManager:
    async def build_context(self, session: CallSession) -> ConversationContext:
        # 1. STM: Contexto inmediato de la sesión actual
        stm = await self.redis.get(f"stm:{session.session_id}")

```



```
# 2. Episodic: ¿Ha llamado antes este caller?
if session.caller_id:
    history = await self.db.get_caller_history(
        session.caller_id, limit=3
    )
    # Resumen comprimido: ~50 tokens por sesión previa
    episode_summary = self.summarizer.compress(history)

# 3. Semantic: Recuperar conocimiento relevante
if session.current_query:
    docs = await self.vector_db.search(
        session.current_query, top_k=3, threshold=0.8
    )

return ConversationContext(
    short_term=stm,
    episodes=episode_summary,      # "Cliente llamó 2 veces antes..."
    knowledge=docs,                # Docs relevantes
    total_tokens=self._count(...)  # Budget: max 500 tokens
)
```

3.4 Sliding Window con Compresión Semántica

TURNO 1-5: [Completo en contexto]
 TURNO 6-10: [Resumidos: "Usuario preguntó por laptops, se informó precio"]
 TURNO 11+: [Comprimidos: "Contexto: consulta inventario laptops HP"]

Resultado: Ventana constante de ~300-400 tokens de historial
 vs. crecimiento lineal sin límite

4. Voice AI Profesional

4.1 Problemas Actuales de Audio

Problema	Ubicación	Impacto
Resampling en Python puro	audio_processor.py L13-35	CPU-bound, ~3ms/chunk
Sin VAD	—	Envía silencio a Gemini (tokens desperdiciados)
Queue drops silenciosos	main.py L138 QueueFull: pass	Pérdida de audio sin aviso
Sin jitter buffer	audiosocket_server.py	Glitches en reproducción
ScriptProcessor deprecado	app.js L65	Será removido de browsers

4.2 Mejoras Propuestas

VAD (Voice Activity Detection)

```
import silero_vad # o webrtcvad

class IntelligentVAD:
    """Filtra silencio ANTES de enviar a Gemini."""

    def __init__(self):
        self.model = silero_vad.load()
        self.speech_threshold = 0.5
        self.silence_duration = 0 # ms acumulados
        self.SILENCE_TIMEOUT = 1500 # ms para end-of-utterance

    def process_chunk(self, audio: bytes) -> VADResult:
        confidence = self.model(audio)

        if confidence > self.speech_threshold:
            self.silence_duration = 0
            return VADResult(is_speech=True, send_to_model=True)
        else:
            self.silence_duration += chunk_duration_ms
            if self.silence_duration > self.SILENCE_TIMEOUT:
                return VADResult(is_speech=False, end_of_utterance=True)
            return VADResult(is_speech=False, send_to_model=False)
```

Impacto: Reducción de 30-50% en audio enviado a Gemini → ahorro directo de costos.

Barge-in Avanzado

FLUJO DE INTERRUPCIÓN:

1. Nova está hablando (audio_queue_out tiene datos)
2. VAD detecta voz del usuario
3. Señal de interrupción:
 - a. Vaciar audio_queue_out inmediatamente
 - b. Enviar señal a Gemini (server_content.interrupted)
 - c. Actualizar turn indicator
4. Procesar nuevo input del usuario
5. Latencia target: <100ms desde detección hasta silencio de Nova

AudioWorklet (reemplazo de ScriptProcessor)

```
// Reemplazar ScriptProcessor deprecado por AudioWorklet
class PCMProcessor extends AudioWorkletProcessor {
    process(inputs, outputs) {
        const input = inputs[0][0];
```

```

        if (!input) return true;

        const pcm16 = new Int16Array(input.length);
        for (let i = 0; i < input.length; i++) {
            pcm16[i] = Math.max(-32768, Math.min(32767, input[i] * 32768));
        }
        this.port.postMessage(pcm16.buffer, [pcm16.buffer]);
        return true;
    }
}

```

Resampling con NumPy/SciPy

```

import numpy as np
from scipy.signal import resample_poly

def resample_fast(data: bytes, from_rate: int, to_rate: int) -> bytes:
    """10-50x más rápido que implementación actual."""
    samples = np.frombuffer(data, dtype=np.int16)
    gcd = math.gcd(to_rate, from_rate)
    resampled = resample_poly(samples, to_rate // gcd, from_rate // gcd)
    return resampled.astype(np.int16).tobytes()

```

Jitter Buffer para AudioSocket

```

class JitterBuffer:
    """Buffer adaptativo para suavizar variaciones de red."""

    def __init__(self, target_ms=60, max_ms=200):
        self.buffer = []
        self.target_packets = target_ms // 20 # 20ms por paquete
        self.max_packets = max_ms // 20

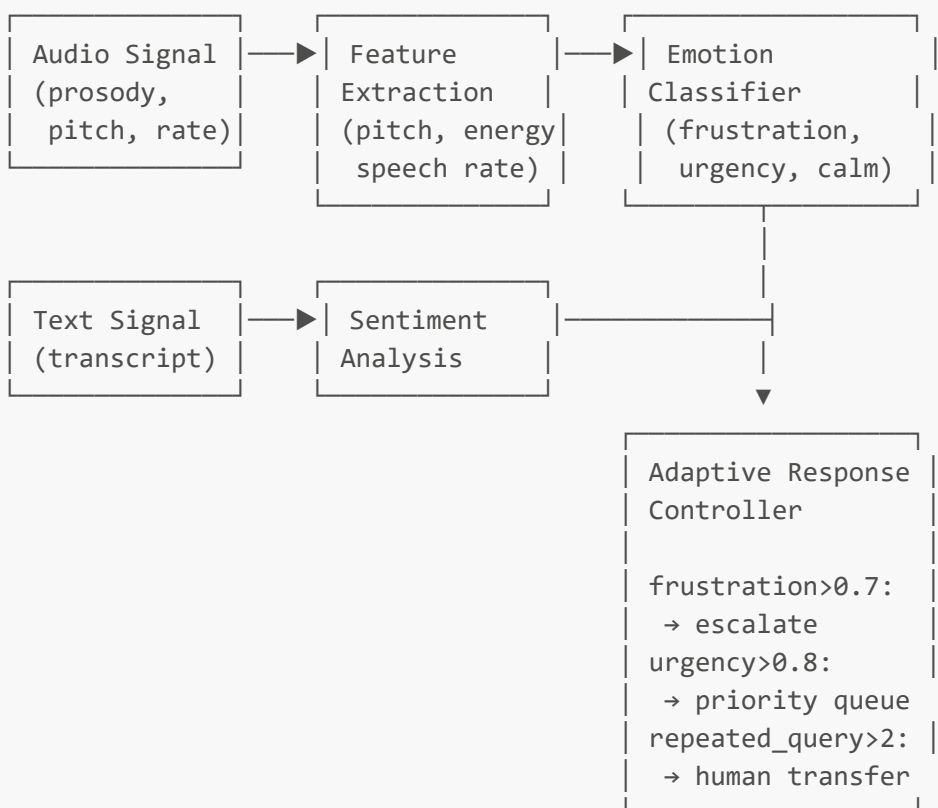
    async def add(self, packet: bytes):
        self.buffer.append(packet)
        if len(self.buffer) > self.max_packets:
            self.buffer.pop(0) # Drop oldest

    async def get(self) -> bytes | None:
        if len(self.buffer) >= self.target_packets:
            return self.buffer.pop(0)
        return None # Buffer underrun → reproducir comfort noise

```

5. Detección Emocional y Comportamiento Adaptativo

5.1 Pipeline de Detección Emocional



5.2 Señales de Frustración

```

class FrustrationDetector:
    SIGNALS = {
        "repeated_intent": 0.3,      # Mismo intent 3+ veces
        "increasing_volume": 0.2,    # Energía audio creciente
        "speech_rate_increase": 0.15, # Habla más rápido
        "negative_keywords": 0.25,   # "no sirve", "otra vez"
        "long_silence_after_ai": 0.1, # No responde tras AI
    }

    def calculate(self, session: CallSession) -> float:
        score = 0.0
        if session.repeated_intents >= 3:
            score += self.SIGNALS["repeated_intent"]
        if session.avg_volume_trend > 1.3:
            score += self.SIGNALS["increasing_volume"]
        # ...
        return min(1.0, score)
  
```

5.3 Escalation Decision Matrix

Frustration	Repeated Queries	Action
< 0.3	< 2	Continuar normal
0.3-0.6	2-3	Aumentar empatía, ofrecer alternativas
0.6-0.8	3+	Ofrecer transferencia a humano proactivamente
> 0.8	cualquier	Transferir automáticamente + prioridad alta

5.4 Adaptive Tone System

```
class AdaptiveToneController:
    """Modifica el prompt dinámicamente según estado emocional."""

    TONE_MODIFIERS = {
        "empathetic": "Muestra empatía genuina. Reconoce la frustración.",
        "urgent": "Sé directo y eficiente. No hagas preguntas innecesarias.",
        "reassuring": "Transmite calma y seguridad. Confirma que estás ayudando.",
        "apologetic": "Discúlpate sinceramente por la inconveniencia.",
    }

    def get_modifier(self, frustration: float, urgency: float) -> str:
        if frustration > 0.7:
            return self.TONE_MODIFIERS["apologetic"]
        if urgency > 0.8:
            return self.TONE_MODIFIERS["urgent"]
        if frustration > 0.4:
            return self.TONE_MODIFIERS["empathetic"]
        return ""
```

6. Arquitectura RAG Empresarial

6.1 Estado Actual: Búsqueda Limitada

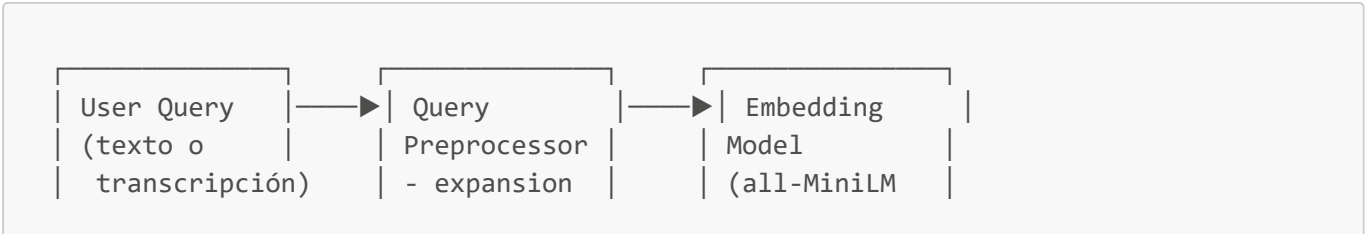
ACTUAL: FTS5 SQLite → token matching → resultados planos

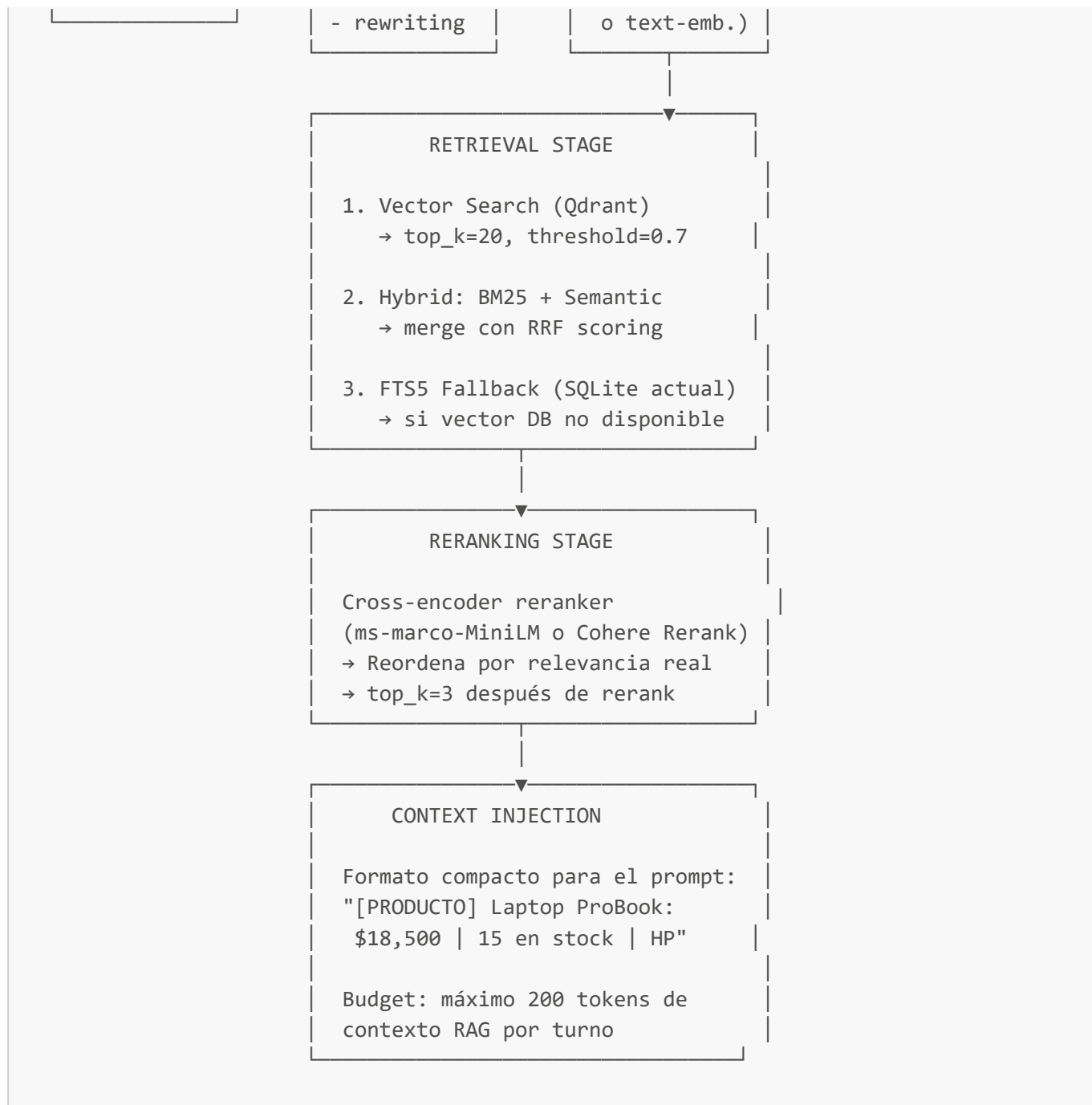
PROBLEMA: No entiende semántica ("pantalla" ≠ "monitor" sin reglas explícitas)

No escala a miles de productos/documentos

No soporta documentos complejos (PDFs, manuales)

6.2 Pipeline RAG Propuesto





6.3 Selección de Vector Database

DB	Pros	Contras	Recomendación
Qdrant	Rust-native, rápido, filtros, local/cloud	Menos maduro	✅ Recomendado para Nova
Weaviate	GraphQL, multi-modal	Más recursos	Buena alternativa
Pinecone	Serverless, fácil	Vendor lock-in, costo	Para MVP rápido
ChromaDB	Simple, Python-native	No para producción	Solo desarrollo
pgvector	Integra con PostgreSQL	Rendimiento limitado	Si ya usas PG

Recomendación: Qdrant (local en Docker para desarrollo, Qdrant Cloud para producción).

6.4 Chunking Strategy

```
class IntelligentChunker:
    """Chunking adaptativo según tipo de contenido."""

    STRATEGIES = {
        "inventory": {
            # 1 chunk = 1 producto completo (nunca partir un producto)
            "method": "record_based",
            "template": "{name} | {description} | ${price} | Stock: {stock} | {category}"
        },
        "extensions": {
            # 1 chunk = 1 extensión con contexto departamental
            "method": "record_based",
            "template": "{name} | Ext: {extension} | {department}"
        },
        "documents": {
            # Chunks semánticos de ~200 tokens con overlap de 50
            "method": "semantic",
            "chunk_size": 200,
            "overlap": 50,
            "separator": "paragraph"
        },
        "faq": {
            # 1 chunk = 1 pregunta-respuesta completa
            "method": "qa_pair",
            "template": "P: {question}\nR: {answer}"
        }
    }
```

6.5 Impacto Económico del RAG

SIN RAG (actual):

Gemini procesa TODA la búsqueda + razonamiento
~150-300 tokens por tool call response
Latencia: ~800ms (Gemini piensa + busca)

CON RAG:

Contexto pre-inyectado: ~50-100 tokens (ya filtrado)
Gemini solo genera respuesta conversacional
Latencia: ~400ms (RAG: 50ms + Gemini: 350ms)

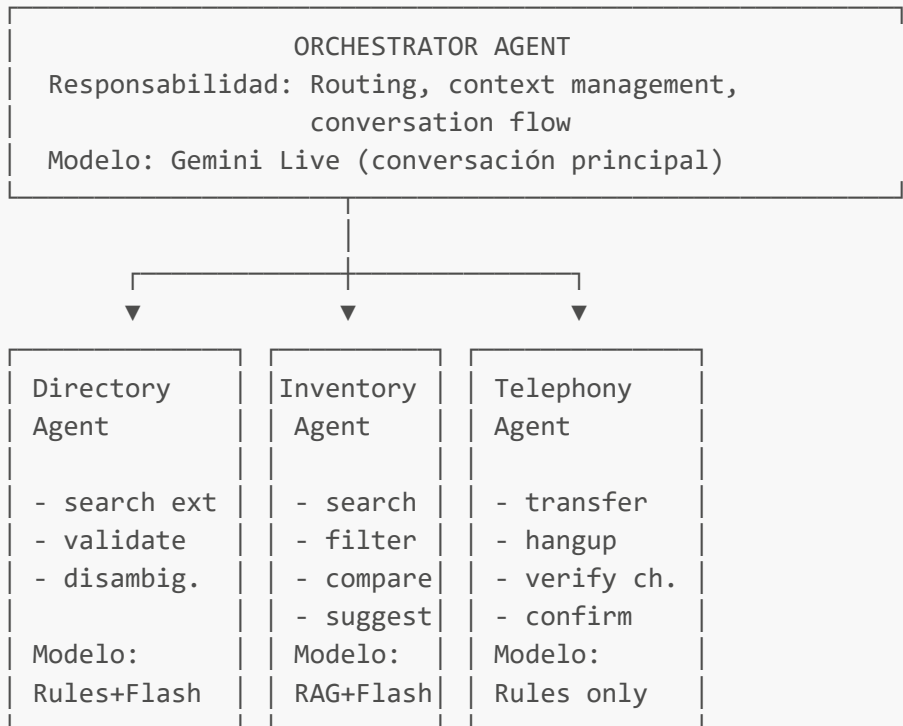
Ahorro por llamada: ~40% tokens de tool responses

7. Tool Calling y Multi-Agent Systems

7.1 Problemas Actuales del Tool System

1. **Sin validación pre-ejecución:** Gemini puede invocar `transfer_call` con datos incorrectos
2. **Sin confirmación de acciones destructivas:** Transfer se ejecuta sin verificar
3. **Sin retry inteligente:** Si falla, se devuelve error y Gemini decide
4. **Sin timeout:** Tool execution puede bloquear indefinidamente
5. **Sin sandboxing:** Tools tienen acceso directo a DB y AMI

7.2 Arquitectura Multi-Agent Propuesta



7.3 Tool Execution Pipeline con Safety Layers

```

class SafeToolExecutor:
    """Pipeline de ejecución con validación, confirmación y rollback."""

    DANGEROUS_ACTIONS = {"transfer_call", "end_call"}

    async def execute(self, tool_call: ToolCall, session: CallSession) ->
    ToolResult:
        # Layer 1: Schema Validation
        if not self.validate_schema(tool_call):
            return ToolResult(error="Parámetros inválidos")

        # Layer 2: Permission Check
        if tool_call.name in self.DANGEROUS_ACTIONS:
            if not session.metadata.get("user_confirmed"):
                return ToolResult(
                    needs_confirmation=True,
  
```



```

        message=f"¿Confirmas que deseas {tool_call.name}?"
    )

    # Layer 3: Rate Limiting por sesión
    if self.rate_limiter.is_exceeded(session.session_id, tool_call.name):
        return ToolResult(error="Demasiadas solicitudes, intenta en unos segundos")

    # Layer 4: Execution con Timeout
    try:
        result = await asyncio.wait_for(
            self.registry.execute(tool_call.name, tool_call.args),
            timeout=5.0 # 5 segundos máximo
        )
    except asyncio.TimeoutError:
        return ToolResult(error="La operación tardó demasiado")

    # Layer 5: Result Validation
    if not self.validate_result(result):
        return ToolResult(error="Resultado inesperado del sistema")

    # Layer 6: Audit Log
    await self.audit_logger.log(session, tool_call, result)

    return ToolResult(success=True, data=result)

```

7.4 Confirmation Flow para Acciones Críticas

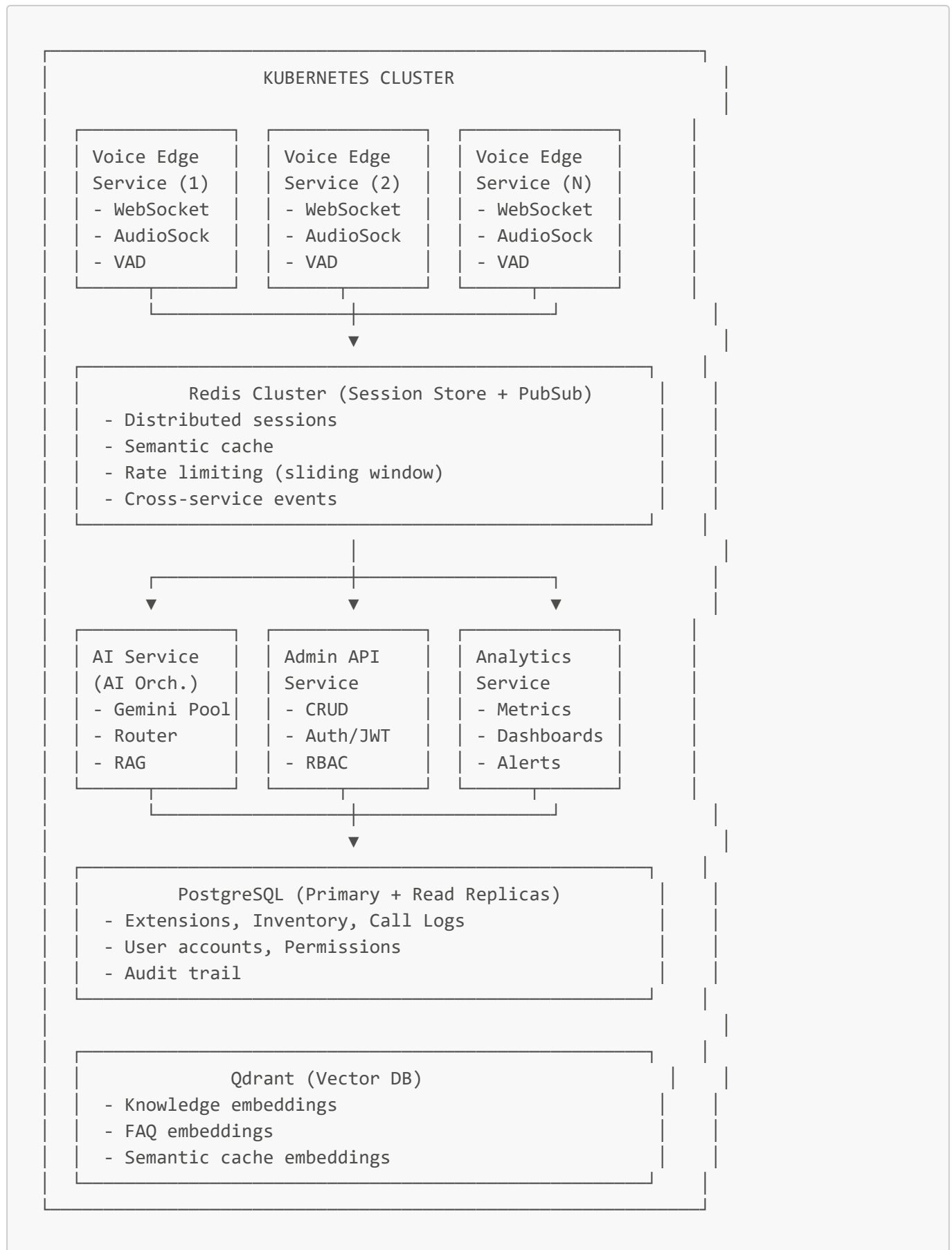
```

USER: "Transfiere a ventas"
|
▼
NOVA: Busca extensión → Encuentra "Ventas (ext 201)"
|
▼
NOVA: "Encontré el departamento de Ventas en la extensión 201.
      ¿Confirmo la transferencia?"
|
▼
USER: "Sí"
|
▼
SYSTEM: session.metadata["user_confirmed"] = True
        → Ejecuta transfer_call
|
▼
NOVA: "Transfiriendo ahora..."

```

8. Escalabilidad Empresarial

8.1 Arquitectura de Microservicios



8.2 Sesiones Distribuidas con Redis

```
class DistributedSessionManager:
    """Sesiones distribuidas en Redis para HA y scaling."""

    async def create_session(self, source: str, **kwargs) -> str:
        session_id = str(uuid.uuid4())
        session_data = {
            "session_id": session_id,
            "source": source,
            "started_at": time.time(),
            "node_id": self.node_id, # Nodo que maneja esta sesión
            "active": True,
            **kwargs
        }
        await self.redis.hset(f"session:{session_id}", mapping=session_data)
        await self.redis.expire(f"session:{session_id}", 3600) # 1h TTL
        await self.redis.sadd("active_sessions", session_id)
        return session_id
```

8.3 Estimaciones de Capacidad

Métrica	Actual (SQLite single)	Target (Microservicios)
Llamadas concurrentes	~5-10	500-1000+
Writes/segundo	~50	10,000+ (PG)
Latencia P99	~1.5s	<800ms
Disponibilidad	~95% (sin HA)	99.9%
Recovery time	Manual restart	<30s (auto-failover)

8.4 Auto-scaling Rules

```
# HPA (Horizontal Pod Autoscaler)
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    name: voice-edge-service
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Pods
    pods:
      metric:
        name: active_voice_sessions
      target:
        type: AverageValue
        averageValue: "15" # Scale up cuando >15 sesiones/pod
  - type: Resource
```

```
resource:
  name: cpu
  target:
    type: Utilization
    averageUtilization: 70
```

9. Seguridad para IA Conversacional

9.1 Vulnerabilidades Actuales

Vulnerabilidad	Riesgo	Estado
Prompt Injection	Usuario podría manipular sistema via voz	Sin protección
API Admin abierta	Cualquiera puede modificar datos	Sin auth
API Key en .env	Exposición si se sube a repo	Parcial
Sin rate limiting	DDoS, abuso de API Gemini	Sin protección
Sin input sanitization	SQL injection posible en búsquedas	Parcial (FTS5)
Audio sin cifrar	Interceptación de conversaciones	HTTP plano

9.2 Prompt Injection Protection

```
class PromptGuard:
    """Detecta y mitiga intentos de prompt injection."""

    INJECTION_PATTERNS = [
        r"ignor[ae]\s+(las|tus)\s+instrucciones",
        r"act[uú]a\s+como",
        r"olvida\s+todo",
        r"system\s*prompt",
        r"(reveal|show|display)\s+(your|the)\s+(prompt|instructions)",
        r"DAN\s+mode",
        r"jailbreak",
    ]

    async def check(self, text: str) -> InjectionResult:
        for pattern in self.INJECTION_PATTERNS:
            if re.search(pattern, text, re.I):
                return InjectionResult(
                    detected=True,
                    action="reject_and_log",
                    response="Lo siento, no puedo hacer eso."
                )

        # Classifier ML para detección avanzada
        score = await self.ml_classifier.predict(text)
```

```

if score > 0.85:
    return InjectionResult(detected=True, action="flag_for_review")

return InjectionResult(detected=False)

```

9.3 Capas de Seguridad Recomendadas

REQUEST FLOW CON SEGURIDAD:

```

Client → [TLS/WSS Termination]
        → [Rate Limiter (Redis: 100 req/min)]
        → [JWT Auth Validation]
        → [Input Sanitization]
        → [Prompt Injection Guard]
        → [Request Size Limits (audio: 10MB/chunk)]
        → [Application Logic]
        → [Tool Execution Sandbox]
        → [Output Validation (no PII leaks)]
        → [Audit Logger]
        → Response

```

9.4 Implementación JWT para Admin API

```

# auth/jwt_handler.py
from jose import jwt, JWTError
from datetime import datetime, timedelta

class JWTManager:
    def __init__(self, secret: str, algorithm: str = "HS256"):
        self.secret = secret
        self.algorithm = algorithm

    def create_token(self, user_id: str, role: str, expires_hours: int = 8):
        payload = {
            "sub": user_id,
            "role": role,
            "exp": datetime.utcnow() + timedelta(hours=expires_hours),
            "iat": datetime.utcnow(),
        }
        return jwt.encode(payload, self.secret, algorithm=self.algorithm)

    def verify(self, token: str) -> dict:
        return jwt.decode(token, self.secret, algorithms=[self.algorithm])

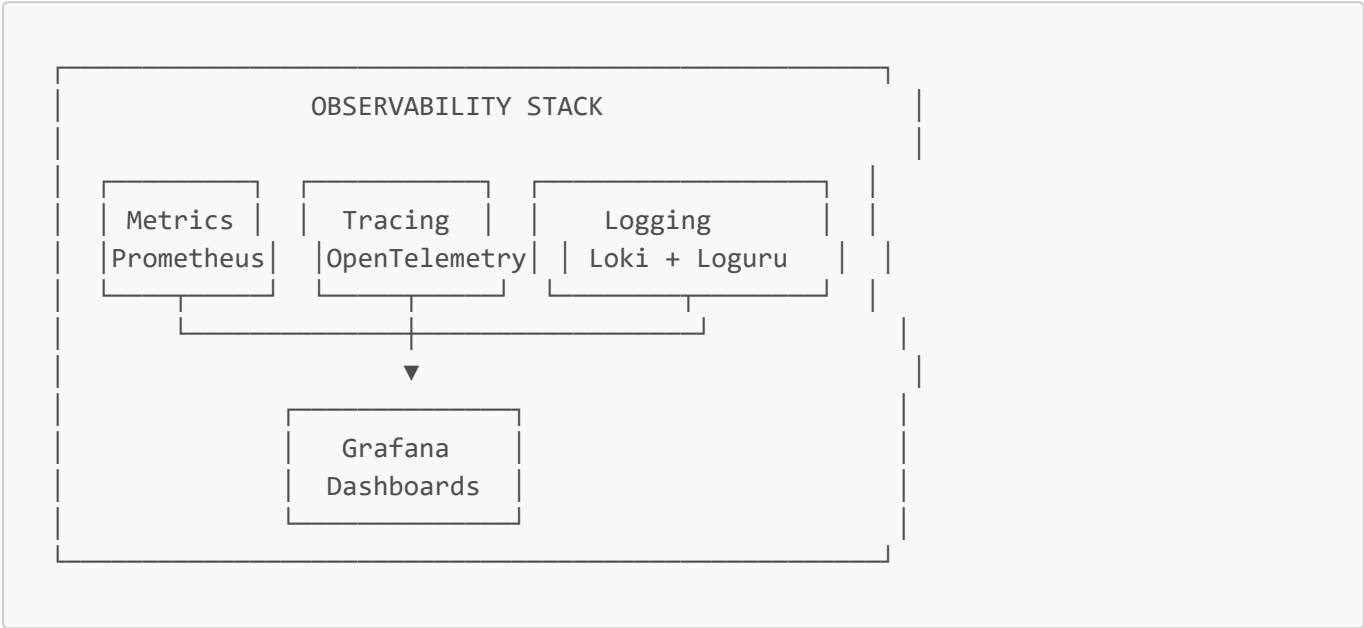
# Middleware para proteger rutas admin
async def require_admin(request: Request):
    token = request.headers.get("Authorization", "").replace("Bearer ", "")
    try:
        payload = jwt_manager.verify(token)

```

```
if payload.get("role") != "admin":
    raise HTTPException(403, "Insufficient permissions")
except JWTError:
    raise HTTPException(401, "Invalid or expired token")
```

10. Sistema de Métricas y Observabilidad

10.1 Stack de Observabilidad



10.2 KPIs Críticos para Nova

Métricas de Negocio

Métrica	Descripción	Target
Call Resolution Rate	% llamadas resueltas sin humano	>75%
Avg Call Duration	Duración promedio de sesión	❤️ min
Transfer Rate	% llamadas transferidas a humano	<25%
First Contact Resolution	Resuelto en primer contacto	>65%
User Satisfaction (CSAT)	Encuesta post-llamada	>4.0/5

Métricas Técnicas

Métrica	Descripción	Target
nova_ai_latency_ms	Tiempo de respuesta Gemini P95	<800ms
nova_tokens_per_call	Tokens consumidos por llamada	<2000

Métrica	Descripción	Target
<code>nova_cost_per_call_usd</code>	Costo IA por llamada	<\$0.10
<code>nova_active_sessions</code>	Sesiones concurrentes	gauge
<code>nova_tool_calls_total</code>	Tool calls por tipo	counter
<code>nova_tool_errors_total</code>	Errores en tool execution	counter
<code>nova_audio_queue_drops</code>	Paquetes audio descartados	counter
<code>nova_vad_speech_ratio</code>	% tiempo con voz detectada	40-60%
<code>nova_barge_in_count</code>	Interrupciones del usuario	counter
<code>nova_cache_hit_rate</code>	Hit rate del semantic cache	>25%

Métricas de Audio

Métrica	Descripción	Target
<code>nova_audio_rtt_ms</code>	Round-trip audio completo	<1200ms
<code>nova_resampling_time_ms</code>	Tiempo de resampling	<2ms
<code>nova_playback_buffer_underruns</code>	Underruns en playback	<1/min

10.3 Implementación con Prometheus

```
from prometheus_client import Counter, Histogram, Gauge

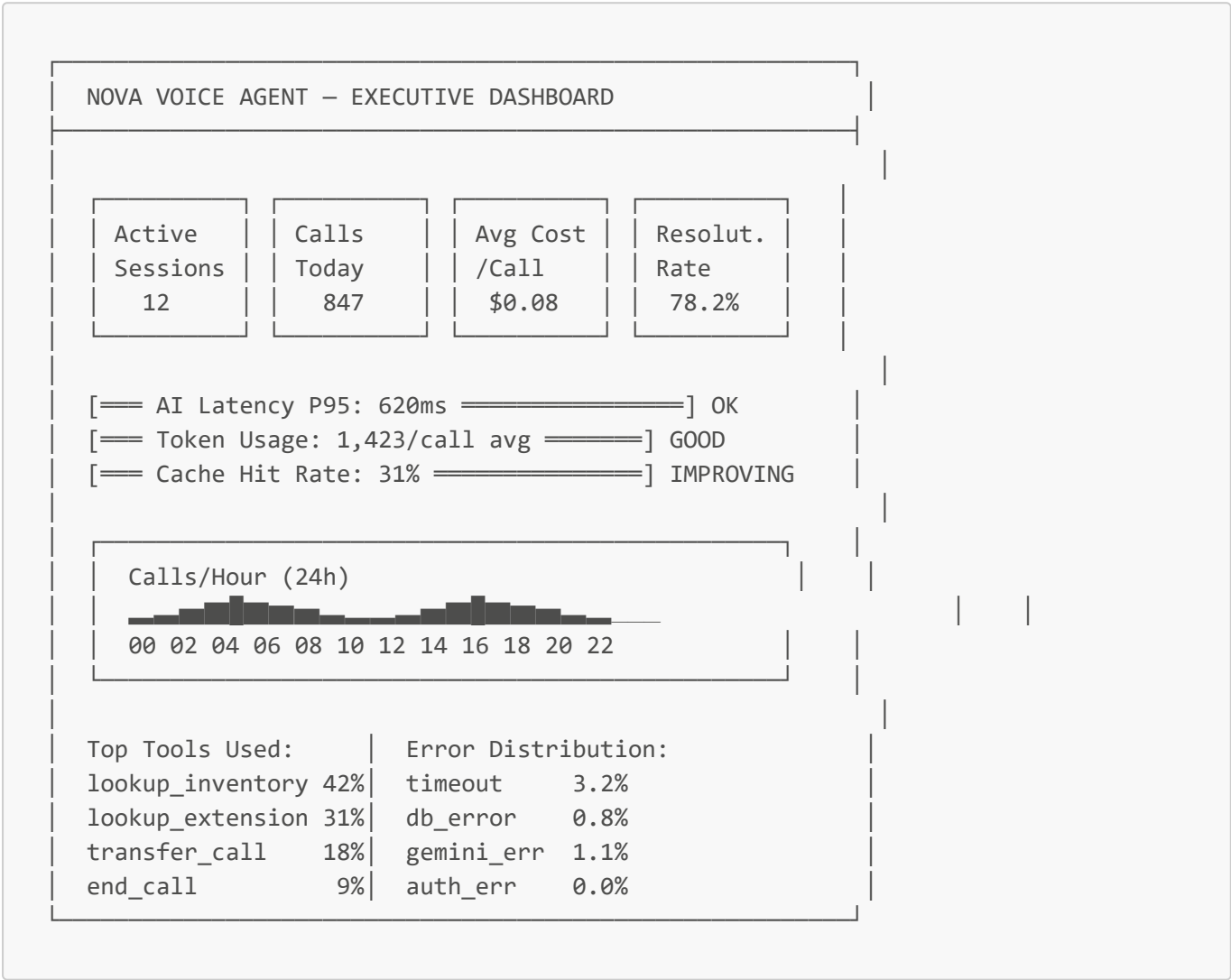
# Métricas de IA
ai_latency = Histogram(
    'nova_ai_latency_seconds', 'AI response latency',
    buckets=[0.1, 0.25, 0.5, 0.75, 1.0, 1.5, 2.0, 5.0]
)
tokens_used = Counter(
    'nova_tokens_total', 'Total tokens consumed',
    ['model', 'direction'] # direction: input/output
)
ai_cost = Counter(
    'nova_ai_cost_usd', 'Estimated AI cost in USD',
    ['model']
)

# Métricas de sesión
active_sessions = Gauge(
    'nova_active_sessions', 'Currently active voice sessions',
    ['source'] # web, asterisk
)
call_duration = Histogram(
    'nova_call_duration_seconds', 'Call duration',
    buckets=[30, 60, 120, 180, 300, 600]
```

```
)

# Métricas de tools
tool_calls = Counter(
    'nova_tool_calls_total', 'Tool invocations',
    ['tool_name', 'status'] # status: success, error, timeout
)
```

10.4 Dashboard Ejecutivo (Grafana)



11. Optimización de Costos

11.1 Desglose de Costo Actual por Llamada

LLAMADA TÍPICA: 3 minutos, 2 tool calls

COMPONENTE	TOKENS/AUDIO	COSTO EST.
System Prompt (input)	~800 tokens	\$0.0001

Audio input (3 min)	~180s audio	\$0.045
Audio output (2 min)	~120s audio	\$0.060
Tool call args (2x)	~100 tokens	\$0.00002
Tool responses (2x)	~300 tokens	\$0.00004
Context/history	~500 tokens	\$0.00006
TOTAL POR LLAMADA		~\$0.105
TOTAL MENSUAL (1000/día)		~\$3,150
TOTAL MENSUAL (5000/día)		~\$15,750

Nota: Precios basados en Gemini 2.5 Flash audio pricing.
Audio input: ~\$0.015/min, Audio output: ~\$0.030/min (estimados).

11.2 Estrategias de Reducción

Estrategia 1: VAD + Audio Filtering (Impacto: -25% costo audio)

Sin VAD: 3 min audio → 3 min procesados → \$0.105
Con VAD: 3 min audio → ~1.8 min (sin silencios) → \$0.068
Ahorro: \$0.037/llamada → \$1,110/mes a 1000 llamadas/día

Estrategia 2: Semantic Cache (Impacto: -15% costos)

Queries frecuentes (horarios, precios populares):
Cache hit: \$0.000 (respuesta local)
Hit rate esperado: 25-35%

Ahorro: ~\$0.026/llamada → \$780/mes a 1000/día

Estrategia 3: Intent Pre-routing (Impacto: -20% costos)

Saludos/despedidas (T0): 15% de interacciones → \$0 c/u
FAQ matches (T1): 20% de interacciones → \$0 c/u
Simple queries (T2 Flash): 30% → \$0.002 c/u (vs \$0.05 Live)

Ahorro promedio: \$0.021/llamada → \$630/mes

Estrategia 4: Context Compression (Impacto: -10% tokens)

Sliding window + resumen:
Sin compresión: ~500 tokens de contexto por turno
Con compresión: ~200 tokens

Ahorro: ~\$0.01/llamada → \$300/mes

11.3 Proyección de Costos Optimizada

COMPARACIÓN MENSUAL (1000 llamadas/día)		
Escenario	\$/mes	Ahorro vs actual
ACTUAL (sin optimizar)	\$3,150	—
+ VAD	\$2,340	-26%
+ VAD + Cache	\$1,890	-40%
+ VAD + Cache + Router	\$1,470	-53%
+ Todo + Compression	\$1,260	-60%
ENTERPRISE OPTIMIZADO	~\$1,260	-60% (\$1,890/mes)

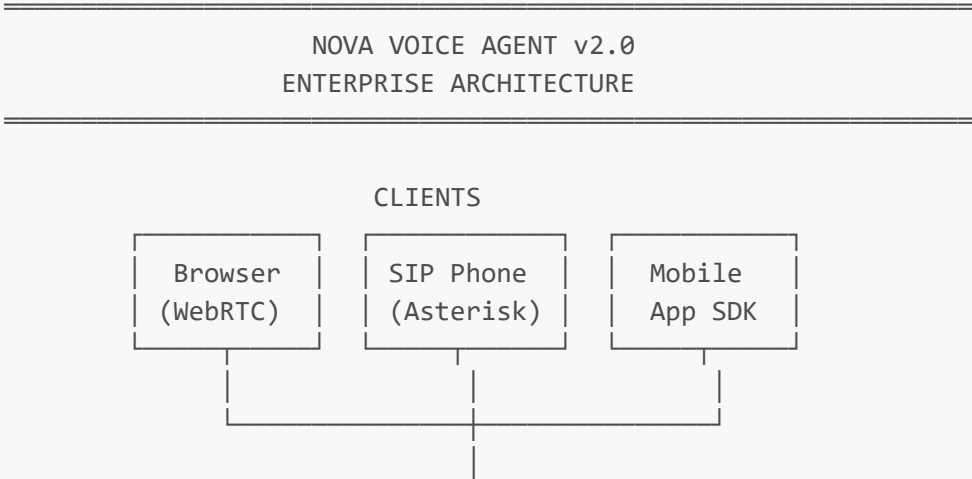
11.4 Modelo de Pricing Sugerido para Clientes

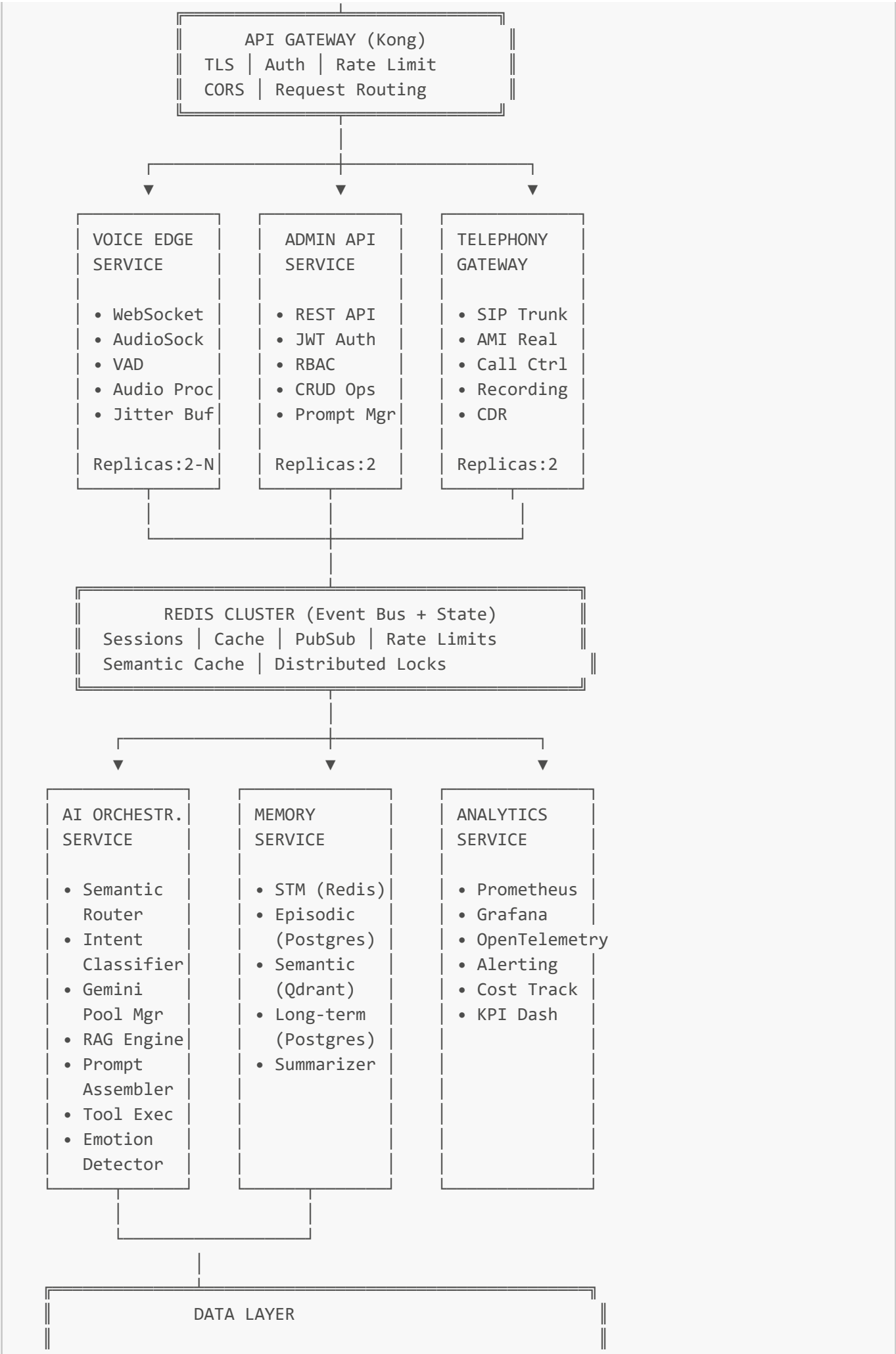
PLAN STARTER: 500 llamadas/mes → \$149/mes (\$0.30/llamada)
PLAN BUSINESS: 2000 llamadas/mes → \$399/mes (\$0.20/llamada)
PLAN ENTERPRISE: 10000+/mes → Custom (\$0.12/llamada)

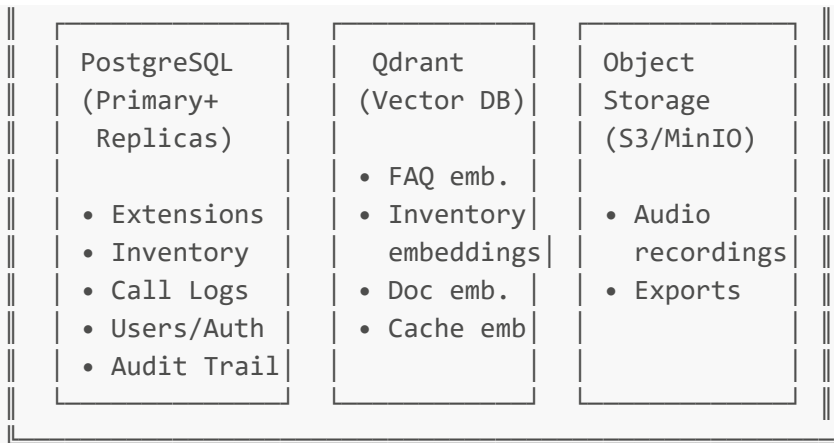
Margen bruto a costo optimizado: 58-77%

12. Arquitectura Final Ideal

12.1 Diagrama Completo Enterprise







12.2 Flujo Conversacional Completo

1. INGRESS
Browser/Phone → API Gateway → Voice Edge Service
2. SESSION INIT
Voice Edge → Redis (create session) → Memory Service (load history)
3. AUDIO PIPELINE
Raw Audio → VAD Filter → Jitter Buffer → Audio Processor
→ Semantic Router (intent pre-classify)
4. AI ROUTING
Router Decision:
 - └ T0 (Rules) → Cached TTS response
 - └ T1 (Embeddings) → RAG → Cached/Template response
 - └ T2 (Flash) → Gemini Flash → Text → TTS
 - └ T3 (Live) → Gemini Live (full audio pipeline)
5. TOOL EXECUTION
Gemini tool_call → Safety Validator → Tool Executor
→ DB/AMI/External → Result Compression → Gemini
6. RESPONSE
AI Response → Audio Output → Jitter Buffer → Client
+ Async: Metrics emit, Memory update, Audit log
7. SESSION END
Hangup/Timeout → Memory summarize → Log persist → Cleanup

13. Roadmap Técnico Profesional

13.1 Quick Wins (1-2 semanas | Alto impacto, baja dificultad)

#	Mejora	Impacto	Dificultad	Beneficio
1	Migrar ScriptProcessor → AudioWorklet	Compatibilidad futura	Baja	Elimina deprecation warnings
2	Resampling con NumPy	-80% CPU en audio	Baja	<code>pip install numpy</code> , 20 líneas
3	Implementar JWT en admin API	Seguridad crítica	Baja	Cerrar vulnerabilidad abierta
4	Comprimir tool responses	-40% tokens tools	Baja	Solo devolver campos relevantes
5	Agregar HTTPS/WSS	Seguridad audio	Baja	Configurar SSL en uvicorn
6	Rate limiting básico	Anti-abuso	Baja	slowapi o middleware custom
7	Health check mejorado	Operabilidad	Baja	Incluir status de Gemini, DB, AMI

13.2 Mejoras Críticas (2-6 semanas | Alto impacto, media dificultad)

#	Mejora	Impacto	Dificultad	Beneficio
8	Silero VAD server-side	-30% costo audio	Media	Filtrar silencio antes de Gemini
9	Semantic caching (Redis)	-15% costos	Media	Cache responses frecuentes
10	Dependency Injection container	Testabilidad	Media	Eliminar globals, facilitar testing
11	Circuit breaker para Gemini	Resiliencia	Media	Manejar caídas de API sin crash
12	AMI real con panoramisk	Funcionalidad	Media	Habilitar transferencias reales
13	Métricas Prometheus básicas	Observabilidad	Media	Latencia, tokens, sesiones
14	Audit logging	Compliance	Media	Registrar todas las acciones

13.3 Mejoras Mediano Plazo (1-3 meses | Transformacional)

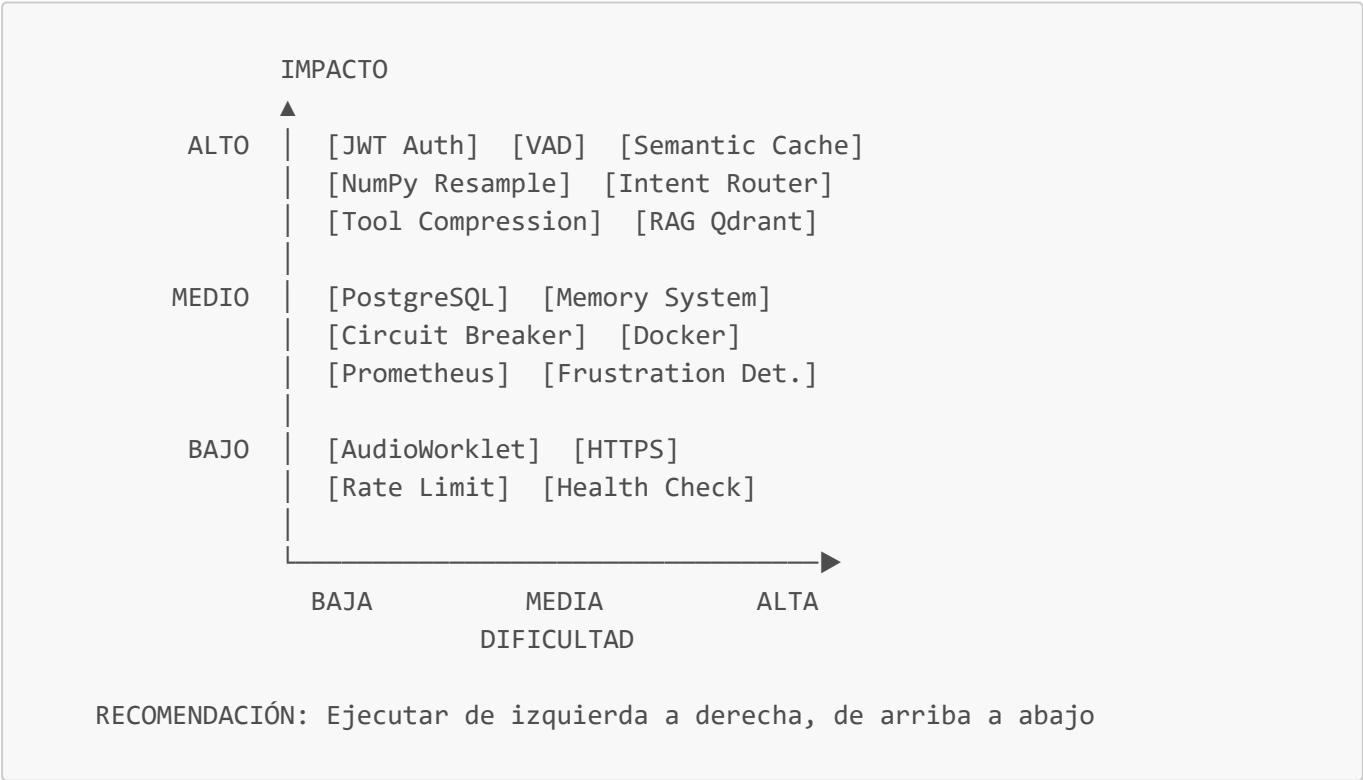
#	Mejora	Impacto	Dificultad	Beneficio
15	Migrar SQLite → PostgreSQL	Escalabilidad	Media-Alta	Concurrencia real, replicación
16	Intent pre-classifier	-20% costos	Alta	Routing inteligente T0/T1/T2
17	RAG con Qdrant	Precisión búsqueda	Alta	Búsqueda semántica real
18	Memory system (episodic)	UX superior	Alta	Continuidad entre sesiones

#	Mejora	Impacto	Dificultad	Beneficio
19	Prompt injection protection	Seguridad IA	Media	Proteger contra manipulación
20	Frustration detection básica	UX	Media	Escalación inteligente
21	Docker + docker-compose	Deployment	Media	Reproducibilidad, portabilidad
22	Test suite automatizada	Calidad	Alta	CI/CD, regression prevention

13.4 Mejoras Enterprise (3-6 meses | Enterprise-grade)

#	Mejora	Impacto	Dificultad	Beneficio
23	Microservicios	Escalabilidad	Muy Alta	Scaling independiente
24	Kubernetes deployment	HA, autoscaling	Muy Alta	99.9% uptime
25	Multi-agent architecture	Modularidad IA	Alta	Agentes especializados
26	OpenTelemetry tracing	Debug distribuido	Media	Tracing end-to-end
27	Grafana dashboards	Business insight	Media	KPIs en tiempo real
28	Multi-tenant support	Comercial	Muy Alta	Múltiples clientes
29	Emotion detection ML	UX premium	Alta	Detección en tiempo real
30	Recording + transcription	Compliance	Media	Auditoría de llamadas

13.5 Priorización Visual



14. Conclusión Técnica Profesional

Evaluación como Principal AI Engineer

Nivel Actual del Proyecto: **POC Avanzado / MVP Pre-Producción**

Nova Voice Agent es un proyecto con una **base arquitectónica sorprendentemente sólida** para su nivel de madurez. La separación modular, el uso correcto de asyncio, el sistema de eventos desacoplado, y la integración con Gemini Live demuestran comprensión real de sistemas de voz en tiempo real.

Fortalezas Genuinas

- **Arquitectura modular real** — no es un script monolítico, tiene separación legítima de concerns
- **Gemini Live integration** — uso correcto de la API bidireccional de audio con tool-calling
- **AudioSocket protocol** — implementación funcional de protocolo binario para Asterisk
- **Prompt Builder system** — sistema de personalidad configurable desde UI admin
- **FTS5 con fallback** — estrategia de búsqueda robusta con degradación graceful
- **EventBus** — patrón pub/sub que facilita extensión futura

Gaps Críticos para Producción

1. **Seguridad:** Admin API abierta, sin auth, sin HTTPS — **bloqueante para deploy**
2. **Escalabilidad:** SQLite + single process = máximo ~10 llamadas concurrentes
3. **Observabilidad:** Cero métricas, cero tracing — operación ciega
4. **Resiliencia:** Sin circuit breaker, retry, fallback — frágil ante fallos

Potencial Comercial: **ALTO** (con las mejoras correctas)

El mercado de Voice AI Agents empresariales está en crecimiento acelerado:

- Competidores: Bland AI (~\$0.09/min), Vapi (~\$0.05/min), Retell AI (~\$0.10/min)
- Nova tiene ventaja diferenciadora: **integración nativa con PBX/Asterisk** (los competidores usan SIP trunking externo)
- Con las optimizaciones propuestas, el costo por llamada de Nova (\$0.04-0.06/min optimizado) sería **competitivo**

Viabilidad Empresarial

SCORING (1-10):		
Arquitectura Base:	7/10	(buena separación, necesita DI y microservicios)
Calidad de Código:	7/10	(clean, async-first, pero globals y sin tests)
IA/NLU:	8/10	(Gemini Live es potente, falta routing)
Voice Quality:	6/10	(funcional, pero sin VAD ni jitter buffer)
Seguridad:	3/10	(vulnerable, auth incompleta)
Escalabilidad:	3/10	(SQLite, single process)
Observabilidad:	2/10	(solo logs)
UX Conversacional:	7/10	(barge-in, turn-taking, buena UI)
Producción-Ready:	4/10	(necesita las mejoras de este roadmap)

Potencial Comercial: 8/10 (mercado en crecimiento, diferenciación real)

SCORE GLOBAL ACTUAL: 5.5/10

SCORE PROYECTADO

(post-roadmap): 8.5/10

Recomendación Final

Nova Voice Agent tiene los fundamentos correctos para convertirse en un producto enterprise competitivo. La inversión de 3-6 meses de ingeniería siguiendo este roadmap llevaría el sistema de un POC funcional a un producto desplegable con clientes reales. La diferenciación clave — integración nativa con PBX corporativos + IA conversacional moderna — es un nicho valioso que los competidores SaaS no cubren bien.

La prioridad inmediata es: **(1) Seguridad, (2) VAD + Optimización de costos, (3) Observabilidad, (4) PostgreSQL.** Estos cuatro pilares desbloquean el path a producción.

Análisis realizado sobre el codebase completo de Nova Voice Agent (TestV1_Speech). Todas las recomendaciones están basadas en patrones utilizados en sistemas enterprise de producción como Amazon Connect, Google CCAI, Twilio Flex, y plataformas Voice AI modernas.